

Research Report

PORTFOLIO TASK 1(A)

Unit code: COS40005

Unit Name: Computing Technology Project A

Submission date: 29/03/2026

Student Name: Minh Hoang Duong

Student ID: 104487115

Word Count: 2465 (excluding reference, coversheet, title page, and multiple titles)

1. RESEARCH ON THE BUSINESS DOMAIN

1.1 Domain Overview:

The business domain of this project lies within the intersection between industrial robotics and artificial intelligence, specifically within the emerging field of intelligence automation or intelligence systems.

Industrial robotics refers to the use of programmable robotic systems to perform manufacturing tasks. The predominant classification of robots currently is the stationary robot, which performs repetitive and yet usually precise mechanical and physical tasks – including assembly, welding, material handling, pick-and-place operations. These robots are implemented in many areas of modern industrial automation and mainly concerned with performance in a structured environment (Siciliano, B. et al; 2009). For decades, this domain has operated on the principle of rigid, preprogramed workflow, where every possible scenario must be explicitly envisioned and encoded in advance by skilled engineers.

This sector is experiencing sustainable growth: the worldwide industrial robotics market was valued at around 23.46 billion USD in 2025 and is anticipated to reach 47.11 billion by 2032 with the annual growth rate (CAGR) of 10.5%. In 2024, the automotive industry alone deployed more than 23,000 robots in Europe with Switzerland leading the robot density (3,876 Robots per 10,00 workers) (Rajat Zope, 2025). In China's automotive sectors, 429,500 robots, representing 470 per 10,000 workers, was aided by the government for domestic manufacturer (Rajat Zope, 2025). In addition, healthcare robotic in surgical assistance, rehabilitation, and hospital logistic reached about 16,700 robots globally by 2024.

These figures reflect the scale and industry depth of current robot deployment across sectors and geography, indicating that industrial level automation is no longer an emerging trend but an established reality that is globally accepted.

1.2 Problem statement:

Despite the large scale explained above, the domain faced a fundamental and persistence problem. As explained above, traditional industrial robot operated on rigids, pre-programmed workflow that required explicit definitions of instructions for every possible scenario, performing in a structured environment (Siciliano, B. et al; 2009). Every change in production requirements (a new objective, a new task sequences, a new workflow), demands intervention by a skilled engineer for reprogramming the process. As modern manufacturing environment grown in increasingly complex, variable and customization/modulization-driven, this inflexibility is merely a disadvantage but becoming a bottleneck that limits the scalability and responsiveness of automated system (Wang et al., 2024).

2. RESEARCH ON END USERS

2.1 Overview of End users:

For the purpose of this project, the identified primary end user group of focus is robotic and automation engineers – the technical professionals that responsible for designing, programming, deploying, operating, and maintaining industrial robotic systems within the manufacturing environment.

2.2 Current understanding:

The conventional automated production plant is controlled by robust control systems that require little to no human intervention. Modern production systems are governed and supervised by one or more Programmable Logic Controllers (PLCs) that coordinate the entire process, providing input/output exchange, task sequencing, and action triggering — alongside a group of dedicated control units for individual machines or sub-systems, including pneumatic and hydraulic actuators, tools, and industrial robots. In this arrangement, low-level control logic is still handled locally by the individual machine or sub-system, while the PLC acts as the centralised coordinator (Bilancia et al., 2023).

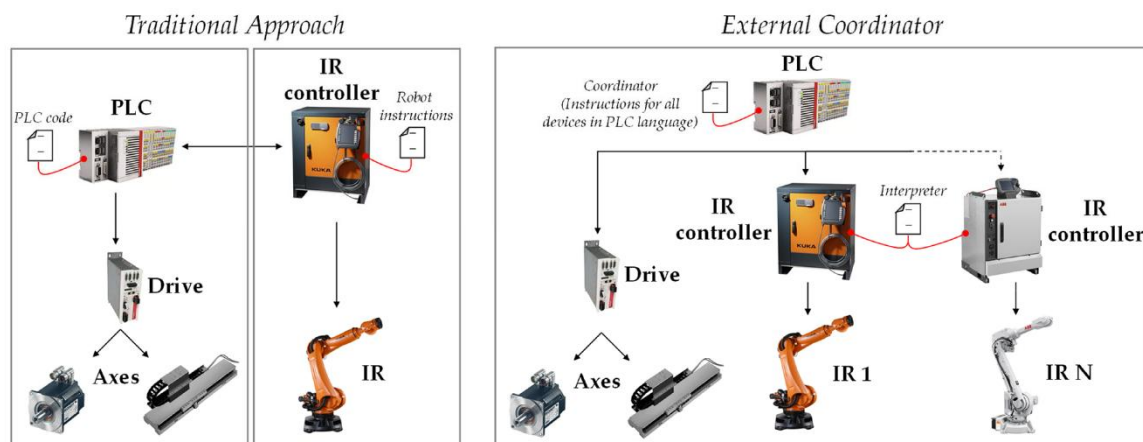


Figure 2.2.o: illustration on current conventional automated manufacturing processes (Bilancia et al., 2023)

PLCs such as Siemens Sinumerik, Bosch IndraMotion, Beckhoff TwinCAT, Mitsubishi MELSEC, and FANUC PMC form the backbone of most industrial automation infrastructure. The commands themselves are typically defined in advance and streamed to the system prior to execution (Bilancia et al., 2023).

A critical structural problem with this architecture is the absence of a universal programming language for industrial robots. Unlike machine tool programming, which is

based on the widely adopted and manufacturer-independent G-code standard, each robot brand is programmed in a completely different manner — ABB uses RAPID, KUKA uses KRL, FANUC uses FANUC TP, and Universal Robots uses URScript — with proprietary languages that differ in complexity, syntax, and semantics, requiring specialised personnel for each platform. Furthermore, robot programmers are constrained to the basic commands and libraries provided by the manufacturer, which cover the majority of standard programming needs but do not permit advanced calculations or elaborate control strategies. Critically, the development of a standardised programming language receives no support from industrial robot vendors, who typically do not disclose detailed information about their low-level control schemas — further entrenching fragmentation across multi-vendor production environments (Bilancia et al., 2023).

Compounding this, the PLC operates on a deterministic scan cycle: it reads all inputs, executes the stored ladder logic or structured text program sequentially, and writes all outputs — repeating this cycle continuously (Collins, 2003). Every possible condition the robot may encounter must be explicitly anticipated and encoded into this logic in advance. When a new product or workflow shift is introduced, the robot requires reprogramming or rebuilding, which takes time, expertise, and expense — and the more integral the robot is to a process, the more reprogramming costs in both dollars and delays.

Taken together, these characteristics reveal a system that is inherently static: optimised for precision and repeatability in fixed, pre-defined environments, but structurally incapable of responding to dynamic instructions or unforeseen task variations without significant human intervention (Siciliano et al., 2016; Wang et al., 2025).

2.3 Gap Addressed:

With the emerging technology of large language models (LLMs) and multimodal AI, introducing an alternative approach. Rather than programming every possible scenario in advance, this system allows robot to received and interpreted high-level natural language instructions, environment through visual perception, and generate executable task plan dynamically (Wang et al., 2024). The ability of LLMs to process vast amount of textual data offer unprecedented potential for enhancing machine's understanding of natural language (Wang et al., 2024). Thanks to the natural language processing capabilities of LLMs, natural language instructions can be translated from input prompts into machine-understandable code that trigger corresponding actions, therefore, rendering the robot more adaptive and flexible, accommodating the increasingly complex and variable demands of the modern industrial environment (Wang et al., 2024). In addition, integrating real world sensor modalities and computer vision (CV) capabilities in LLMs facilities could establish the connection between words, instructions, and perception, rendering the system more capable in various tasks, closing the previously mentioned gap

between the rigid, preprogramed workflow of conventional industrial robots and the dynamic, flexibility demands of modern manufacturing (Wang et al., 2024).

2.4 Project Proposal:

Understanding these key points, this project aimed to develop an intelligence robotic system that integrated large language model (LLMs), computer visions, and task planning mechanisms into a robotic system that can understand natural language instructions, environment perception, and tasks execution.

Objective: The primary objective of this project is to design and implement a multimodal AI pipeline that bridges the gap between human language and robotic action.

3. RESEARCH ON THE SOLUTION DOMAIN

3.1 Programming language:

Python is the most appropriate and explicitly specified programming language for this project. It provides native support for all major AI and robotics libraries used in this system including LangChain, YOLO, OpenCV, Pybullet,...

Python is the chosen programming language for this project.

3.2 Large Language Model and Prompt Engineering Framework

The LLM component is the cognitive core of the system, responsible for receiving natural language instructions, extracting structured intents including object references, actions, and spatial relationships — and producing a step-by-step task plan.

The **OpenAI API** (GPT-4 and successors) is the most capable and best-documented option for this use case. Models like GPT-4 demonstrate exceptional capabilities in zero-shot and few-shot learning, enabling them to understand context from minimal prompts and examples, with recent studies demonstrating their potential in enhancing a robot's ability to understand operational context and adapt its actions. Structured output techniques are used to ensure the model returns consistent, parse able task plans (OpenAI, 2023).

Local LLMs such as **LLaMA**, **Qwen3.5** offer an offline alternative but introduce hardware constraints and reduced capability that are unsuitable for a simulation-focused academic prototype.

For structuring LLM workflows, **LangChain** provides a modular framework for chaining prompts, managing conversation history, and integrating tool calls. LangChain helps developers build LLM-powered applications through a standard interface for models, embeddings, and integrations, enabling rapid prototyping with a modular, component-

based architecture that supports model interoperability — allowing models to be swapped as the team experiments to find the best option (www.langchain.com, n.d.).

3.3 Visual Perception Module

The visual perception module identifies objects, their positions, and spatial relationships within the simulated workspace. Two tools form the core of this module: YOLO for object detection and OpenCV for image processing.

YOLO (You Only Look Once) is the most widely adopted real-time object detection framework in robotics research. Its unified Python package and PyTorch compatibility make integration into the pipeline straightforward. The YOLO family's single-stage detection approach, which treats detection as a direct regression problem without a separate region proposal step, is particularly well-suited to real-time robotic perception where latency must be minimized (Redmon et al. 2015).

OpenCV complements YOLO by providing foundational image processing capabilities — including colour space conversion, contour detection, spatial coordinate extraction, and image transformation — that are necessary for interpreting the visual state of the simulation environment (Bradski, 2000). Together, YOLO and OpenCV form a well-validated and computationally accessible perception stack for academic robotics research.

3.4 Simulation Environment:

The simulation environment provides the execution platform for validating the system's task planning pipeline without the complexity of physical. The primary candidate is PyBullet.

3.5 Existing Architectural Approaches:

SayCan (Ahn et al., 2022) is one of the most cited systems for LLM-driven robotic task planning. In Google's PaLM-SayCan system, a pre-trained LLM (PaLM) is coupled with a set of pre-learned robotic skills and an affordance function. The LLM provides semantic understanding of an open-ended instruction, breaking it into feasible steps, while the affordance model estimates which actions are physically possible in the current context.

PaLM-E (Driess et al., 2023) takes a more deeply integrated approach, injecting visual observations directly into the token sequence of a large language model, enabling the model to jointly reason over language and perception. This architecture removes the boundary between the language and vision modules entirely.

Taken together, these systems reveal a convergent architectural pattern: a language model for high-level reasoning, a vision system for environmental grounding, and a structured intermediate representation that bridges the two. This project operates exactly this pattern using accessible, open-source tools.

3.6 Proposed Architecture

The recommended architecture follows a modular pipeline structure in which each component handles a specific and well-defined responsibility.

Stage 1 — Natural Language Input. The user provides a task instruction in plain English.

Example: “Place the blue block on the right tray.”

Stage 2 — Language Understanding (LLM Module). The instruction is passed to the LLM module, where a large language model — accessed via the OpenAI API — processes the input and extracts structured intent. This includes identifying the objects referenced (e.g., *blue block*), the action required (e.g., *place*), and the spatial relationship specified (e.g., *right tray*).

Stage 3 — Visual Perception Module. Concurrently with language understanding, the system analyses the simulated environment using computer vision.

Stage 4 — Task Planning Module. This is the integration point of the pipeline, where the outputs of the LLM module and the visual perception module are combined. The structured intent extracted from the natural language instruction is merged with the scene description produced by the vision module, and the LLM generates a step-by-step executable action plan. A representative plan for the example instruction above might consist of: locate the blue block → navigate the robot arm to its position → pick up the object → move to the right tray → place the object.

Stage 5 — Execution Module. The generated action plan is executed within the PyBullet simulation environment, where the robot agent performs physical actions including moving, picking, and placing objects.

Stage 6 — Feedback and Validation. After execution, the system evaluates whether the task was completed successfully. If a failure is detected — such as an object not being found, a pick action missing its target, or a placement landing outside the designated area — the system logs the error and may trigger a re-planning cycle.

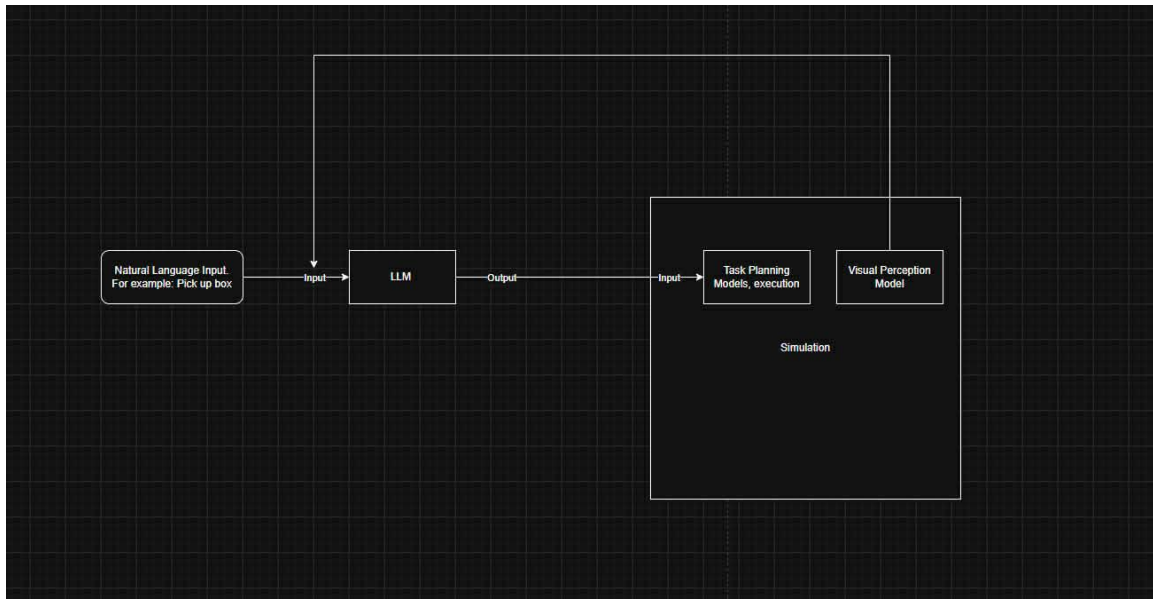


Figure 3.6.o: illustration on high-level proposal architecture.

4. KOST ANALYSIS

The team has good background knowledge and skill of python programming, machine learning, and data analysis, and some basic understanding of large language models and systems like LangChain. The team is also capable of collaborating, communicating, and documenting well, which form critical management aspects in a multidisciplinary project.

However, it is also found that there are a number of knowledge and technical gaps. These encompass the lack of experience in integrating multimodal AI systems, computer vision algorithms like object detection and computer simulators like PyBullet. Besides, the team has little experience in integrating the outputs of LLM with real-time perception of the environment and task planning.

To address these gaps, the team shall embrace a systematic learning process. This also involves the use of online tutorials, research papers, and official documentation of tools like OpenCV, YOLO, and PyBullet. The tasks will be allocated to the team members as per their strengths and weekly meetings will be held to monitor the progress and address the challenges. On-going testing and gradual development will also show to it that every module will be proved before it can be integrated.

5. ETHICAL CONSIDERATIONS

The multimodal AI system used in the planning of industrial tasks brings up several ethical questions concerning safety, reliability, and responsible use of AI. Considering that the system implies the interpretation of the commands in a natural language and their following actions, there is a chance of misinterpretation, which might result in unintentional or hazardous activities in the simulation scenario.

To avoid this, validation mechanisms will be employed so that the task plans that are generated will be checked to be logically correct and safe before being executed. The system will also have error handling measures that will be used to deal with vague or ambiguous instructions.

Data and privacy wise, the system will comply with the Australian Privacy Principles (APP) in the privacy act 1988 (Cth) to make sure that the input of a user is processed in a secure and conscientious way. Though the project is simulation-oriented and there is no risk of working with sensitive personal data, these principles are viewed to ensure the ethics in designing a system.

Moreover, the project will be in line with the bigger regulatory frameworks (Australian AI Ethics Principles, other international guidelines, e.g., EU General Data Protection Regulation (GDPR)) that focus more on transparency, accountability, and responsible data processing.

Moreover, the system will seek to ensure the reduction of bias in decision-making by not allowing outputs to be made with subjective implications on the basis of structured inputs. Among the major AI ethics principles to be followed in the project, there will be fairness, transparency, and reliability to promote responsible system behaviour.

6. AI DECLARATION:

I used Perplexity.ai to assist with researching and looking up documentations, citations along with polishing the provided text based on the provided citations, or research paper. I also used Claude Sonnet 4.6 for coming up with the writing structures, and grammatically, linguistic polishing. All final submission is my own work, verified independently by personal proof reading between the sources, citation and my writing.

7. REFERENCES

1. Sanz, Pedro. (2009). Robotics: Modeling, Planning, and Control (Siciliano, B. et al; 2009) [On the Shelf]. Robotics & Automation Magazine, IEEE. 16. 101-101. 10.1109/MRA.2009.934833.
2. Wang, J., Wu, Z., Li, Y., Jiang, H., Shu, P., Shi, E., Hu, H., Ma, C., Liu, Y., Wang, X., Yao, Y., Liu, X., Zhao, H., Liu, Z., Dai, H., Zhao, L., Ge, B., Li, X., Liu, T. and Zhang, S. (2024). Large Language Models for Robotics: Opportunities, Challenges, and Perspectives. [online] arXiv.org. doi: <https://doi.org/10.48550/arXiv.2401.04334>.
3. Rajat Zope (2025). Industrial Robotics Market Size, Share & Statistics, 2032. [online] Persistence Market Research. Available at: <https://www.persistencemarketresearch.com/market-research/industrial-robotics-market.asp> [Accessed 29 Mar. 2026].
4. Bilancia, P., Schmidt, J., Raffaelli, R., Peruzzini, M. and Pellicciari, M. (2023). An Overview of Industrial Robots Control and Programming Approaches. Applied Sciences, 13(4), p.2582. doi:<https://doi.org/10.3390/app13042582>.
5. Collins, K. (2003). PLC programming for industrial automation. Technicspublications.
6. OpenAI. (2023). GPT-4 technical report. arXiv preprint arXiv:2303.08774.
7. www.langchain.com. (n.d.). LangChain. [online] Available at: <https://www.langchain.com>.
8. Redmon, J, Divvala, S, Girshick, R & Farhadi, A 2015, You only look once: Unified, Real-Time Object Detection, <https://arxiv.org/abs/1506.02640>.
9. Bradski, G. (2000). The OpenCV library. Dr. Dobb's Journal of Software Tools, 25(11), 120–125.

10. Ahn, M, Brohan, A, Brown, N, Chebotar, Y, Cortes, O, David, B, Finn, C, Fu, C, Gopalakrishnan, K, Hausman, K, Herzog, A, Ho, D, Hsu, J, Ibarz, J, Ichter, B, Irpan, A, Jang, E, Ruano, RJ, Jeffrey, K, Jesmonth, S, Joshi, NJ, Julian, R, Kalashnikov, D, Kuang, Y, Lee, K-H, Levine, S, Lu, Y, Luu, L, Parada, C, Pastor, P, Quiambao, J, Rao, K, Rettinghouse, J, Reyes, D, Sermanet, P, Sievers, N, Tan, C, Toshev, A, Vanhoucke, V, Xia, F, Xiao, T, Xu, P, Xu, S, Yan, M & Zeng, A 2022, Do as I can, not as I say: grounding language in robotic affordances, <https://arxiv.org/abs/2204.01691>.